

Lesson 7: Building a Secure File Encryption in Java

Marcin Kurzyna

Lecture Goals

- Generate cryptographically secure random values,
- Create SHA-256 hashes,
- Generate encryption keys,
- Encrypt and decrypt data using AES,
- Understand why AES replaced DES,
- Build a simple secure encryption application.

1 Project Overview

In this lecture we will build a simple security application that:

1. Accepts a user password,
2. Generates a random salt,
3. Computes a SHA-256 hash,
4. Generates an AES key,
5. Encrypts a message,
6. Decrypts the message,
7. Verifies user credentials.

2 Part 1: Secure Random Numbers

Many cryptographic operations require random values. Examples:

- Salts,
- Encryption keys,
- Session identifiers,
- Initialization vectors.

Java provides the `SecureRandom` class.

2.1 Why Not Use Random?

The following class should not be used for cryptographic purposes:

```
import java.util.Random;

Random random =
    new Random();
```

Instead use:

```
import java.security.SecureRandom;

SecureRandom random =
    new SecureRandom();
```

SecureRandom produces much less predictable values.

2.2 Generating a Salt

A salt is a random value added to a password before hashing.

```
import java.security.SecureRandom;

SecureRandom random =
    new SecureRandom();

byte[] salt =
    new byte[16];

random.nextBytes(salt);
```

This creates a random 16-byte salt.

2.3 Displaying Random Bytes

```
for (byte b : salt) {
    System.out.printf(
        "%02X", b);
}
```

Sample output:

```
A71F22B9C8D4E310...
```

Every execution generates a different value.

3 Part 2: Password Hashing with SHA-256

Popular hash algorithms include:

- SHA-256,
- SHA-512,
- SHA-3.

3.1 Creating a SHA-256 Hash

```
import java.security.MessageDigest;

String password =
    "admin123";

MessageDigest md =
    MessageDigest.getInstance(
        "SHA-256");

byte[] hash =
    md.digest(
        password.getBytes());
```

3.2 Converting Hashes to Hex

```
StringBuilder sb =
    new StringBuilder();

for (byte b : hash) {
    sb.append(
        String.format(
            "%02x", b));
}

System.out.println(
    sb.toString());
```

This produces a readable hash value.

3.3 Hashing with Salt

Before hashing, we can include the salt.

```
md.update(salt);

byte[] hash =
    md.digest(
        password.getBytes());
```

Benefits:

- Identical passwords produce different hashes,
- Rainbow tables become ineffective,
- Password attacks become harder.

4 Part 3: Key Generation

Encryption algorithms require cryptographic keys. Without the correct key, encrypted data cannot be decrypted.

4.1 Generating an AES Key

```
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;

KeyGenerator generator =
    KeyGenerator.getInstance(
        "AES");

generator.init(128);

SecretKey key =
    generator.generateKey();
```

This creates a 128-bit AES key.

4.2 Examining the Key

```
System.out.println(
    key.getEncoded().length);
```

Output:

```
16
```

16 bytes equals 128 bits.

5 Part 4: AES Encryption

AES is currently the most widely used symmetric encryption algorithm.

Advantages:

- Fast,
- Secure,
- Industry standard,
- Supported by Java.

5.1 Encrypting a Message

```
import javax.crypto.Cipher;

Cipher cipher =
    Cipher.getInstance("AES");

cipher.init(
    Cipher.ENCRYPT_MODE,
    key);

byte[] encrypted =
    cipher.doFinal(
        "Hello World".getBytes());
```

The encrypted bytes are unreadable.

5.2 Displaying Ciphertext

Because encrypted data is binary, we often use Base64.

```
import java.util.Base64;

String encoded =
    Base64.getEncoder()
        .encodeToString(
            encrypted);

System.out.println(
    encoded);
```

Example output:

```
t3A6Rw5Q9v2P5a7X...
```

Decrypting a Message

```
cipher.init(
    Cipher.DECRYPT_MODE,
    key);

byte[] decrypted =
    cipher.doFinal(
        encrypted);

System.out.println(
    new String(
        decrypted));
```

Output:

```
Hello World
```

Complete AES Example

```
import javax.crypto.*;
import java.util.Base64;

public class AESDemo {

    public static void main(String[] args)
        throws Exception {

        KeyGenerator generator =
            KeyGenerator.getInstance(
                "AES");

        generator.init(128);

        SecretKey key =
            generator.generateKey();
```

```

Cipher cipher =
    Cipher.getInstance(
        "AES");

String message =
    "Java Cryptography";

cipher.init(
    Cipher.ENCRYPT_MODE,
    key);

byte[] encrypted =
    cipher.doFinal(
        message.getBytes());

String encoded =
    Base64.getEncoder()
        .encodeToString(
            encrypted);

System.out.println(
    encoded);

cipher.init(
    Cipher.DECRYPT_MODE,
    key);

byte[] decrypted =
    cipher.doFinal(
        encrypted);

System.out.println(
    new String(
        decrypted));
}
}

```

5.3 DES vs AES

DES	AES
56-bit key	128/192/256-bit key
Obsolete	Modern standard
Vulnerable to attacks	Secure
Rarely used today	Widely used

6 Generating a DES Key

```

KeyGenerator generator =
    KeyGenerator.getInstance(

```

```
        "DES");  
  
SecretKey key =  
    generator.generateKey();
```

Modern systems should use AES.

7 Final Project: Secure Vault

The application should:

1. Read a password,
2. Generate a random salt,
3. Create a SHA-256 hash,
4. Generate an AES key,
5. Encrypt a message,
6. Display encrypted data,
7. Decrypt and display the original message.

Expected output:

```
Password:  
admin123  
  
Salt:  
A71F22B9C8...  
  
Hash:  
7f45c8ab1d...  
  
Encrypted:  
t3A6Rw5Q9v2...  
  
Decrypted:  
Hello World
```

Summary

In this lecture we learned:

- Secure random number generation,
- Password hashing with SHA-256,
- Salts,
- AES key generation,

- AES encryption,
- AES decryption,
- Differences between AES and DES.

8 Exercises

1. Explain why SecureRandom is preferred over Random.
2. What is the purpose of a salt?
3. Explain the difference between hashing and encryption.
4. Why should passwords never be stored in plain text?
5. Why is AES preferred over DES?
6. Generate a random 32-byte value using SecureRandom.
7. Create a SHA-256 hash of your name.
8. Create a SHA-256 hash of your student ID.
9. Generate a 128-bit AES key.
10. Generate a 256-bit AES key.
11. Encrypt the following message using AES:

```
Java Security
```
12. Decrypt the encrypted message.
13. Convert encrypted data into Base64 format.
14. Create two different salts and hash the same password using both salts.
Compare the results.
15. Explain why the hashes are different.
16. Challenge:
Build the complete Secure Vault application described in this lecture.
The program should:
 - Generate a salt,
 - Hash a password,
 - Generate an AES key,
 - Encrypt a message,
 - Decrypt the message,
 - Display all intermediate results.